

2:05 PM

Machine Test

Question1: You are required to host a simple web page on an EC2 instance using a NEW VPC and then integrate the setup with a Load Balancer and Auto Scaling to make the application highly available and scalable.

Additional: Better if you can configure it using CICD using code deploy any new changes should deploy automatically into server when any changes are made on git (use your own git account)

Question 2:

Docker Machine Test (90 Minutes)

Scenario

You are asked to containerize a simple 3-tier application:

Browser → Nginx → Node App → MySQL

Task 1 – Multi-Container Application

Provided:

Node App (app.js)

```
const express = require("express");
```

```
const mysql = require("mysql2");
```

```
const app = express();
```

```
const db = mysql.createConnection({
```

```
  host: process.env.DB_HOST,
```

```
  user: "root",
```

```
  password: "root",
```

```
  database: "testdb"
```

```
});
```

```
app.get("/", (req, res) => {  
  db.query("SELECT NOW() as time", (err, result) => {  
    if (err) return res.send("DB Error");  
    res.send("DB Time: " + result[0].time);  
  });  
});
```

```
app.listen(3000);
```

Requirements:

1. Create Dockerfile for Node app
2. Use **Alpine base image**
3. Use non-root user
4. Use environment variables
5. Expose port 3000

Task 2 – Docker Compose Setup

Create docker-compose.yml:

- Node service
- MySQL service
- Nginx reverse proxy

Requirements:

- MySQL with persistent volume
- Use custom bridge network

- Environment variables via .env
- Nginx reverse proxy to Node
- Restart policy enabled

Validation:

<http://localhost>

Task 3 – Data Persistence Test

1. Insert a record manually inside the DB
2. Restart containers
3. Prove data still exists

Task 4 – Optimization

Improve Dockerfile:

- Reduce image size
- Use multi-stage build
- Add .dockerignore